Development Testing (الاختبار أثناء التطوير)

تعريف Development Testing

Development Testing = كل أنشطة الاختبار التي يُجريها فريق التطوير نفسه أثناء بناء النظام — هدفه الرئيسي اكتشاف الأخطاء (**Defect Testing**) وهو مرتبط دائماً بـ **Debugging**.

Debugging = عملية تحديد موقع الخطأ وإصلاحه بعد اكتشافه بالاختبار.

الهيكل العام — 3 مستويات

- **8.1.1 Unit Testing** ← اختبار الوحدة الفردية
- **8.1.2 Choosing Test Cases** ← كيف تختار حالات الاختبار
- **8.1.3 Component Testing** ← اختبار المكوّنات المدمجة
- **8.1.4 System Testing** ← اختبار النظام كاملاً

---

1 — Unit Testing

Unit Testing = اختبار وحدات برمجية فردية — دوال، طرق، أو كلاسات كاملة.

عند اختبار Object Class يجب تغطية 3 أشياء:

ما الذي تختبره؟	المثال
جميع العمليات (Operations)	reportWeather(), reportStatus(), shutdown()
جميع الخصائص (Attributes)	تعيين قيمة identifier والتحقق منها
جميع الحالات (States)	محاكاة كل الأحداث التي تُسبب تغيير الحالة

⚠ تحذير مهم: الوراثة (Inheritance)

الوراثة تُعقّد اختبار الكلاسات — لا يكفي اختبار العملية في الكلاس الأصل فقط، بل يجب اختبارها في كل كلاس فرعي يرثها لأن الافتراضات قد لا تنطبق عليها.

مثال: تسلسلات حالات Weather Station للاختبار

- Shutdown ← Running ← Shutdown
- Configuring ← Running ← Testing ← Transmitting ← Running
- Running ← Collecting ← Running ← Summarizing ← Transmitting ← Running

الاختبار التلقائي (Automated Unit Testing)

أداة **JUnit** = أشهر إطار عمل لأتمتة الاختبارات في Java.

الجزء	الوصف
1. Setup	تهيئة النظام بالمدخلات والمخرجات المتوقعة
2. Call	استدعاء الكائن أو الطريقة المراد اختبارها
3. Assertion	فشل = false /نجاح = true — مقارنة النتيجة الفعلية بالمتوقعة

Mock Objects 🎭

Mock Objects = كائنات وهمية بنفس واجهة الكائنات الحقيقية تُحاكي وظيفتها.

متى تُستخدم؟	المثال
الكائن المعتمد عليه بطيء	Mock لقاعدة البيانات يحتوي بيانات قليلة في مصفوفة بدل استدعاء الـ DB
محاكاة أحداث نادرة	Mock للساعة يُرجع وقتاً محدداً بدل انتظار الوقت الفعلي

---

2 — Choosing Unit Test Cases

الهدف: اختيار حالات اختبار فعّالة — توفر التغطية وتكشف العيوب.

استراتيجيتان أساسيتان:

الاستراتيجية	الوصف
Partition Testing	تقسيم المدخلات لمجموعات بنفس السلوك
Guideline-Based Testing	استخدم إرشادات مبنية على تجارب سابقة لأنواع الأخطاء الشائعة

أ) Equivalence Partitioning (التقسيم المتكافئ)

المبدأ: البرنامج يتصرف بنفس الطريقة مع كل أعضاء نفس المجموعة ← اختبار عضو واحد كافٍ لتمثيل المجموعة. المجموعات تُسمى: **Equivalence Partitions** أو **Domains**.

مثال تطبيقي من الكتاب:

البرنامج يقبل 4 إلى 8 مدخلات من الأرقام الصحيحة الخمسية < 10,000

الـ Partition	نوعه	قيم الاختبار
أرقام بين 10000 و 99999	صحيح ✅	9999 ، 10000 ، 50000 ، 99999 ، 100000
أرقام أقل من 10000	خطأ ❌	9999
أرقام أكثر من 99999	خطأ ❌	100000
عدد مدخلات بين 4 و 8	صحيح ✅	3 ، 4 ، 7 ، 10 ، 11
عدد مدخلات أقل من 4	خطأ ❌	3
عدد مدخلات أكثر من 8	خطأ ❌	11

القاعدة الذهبية: اختر قيماً على حدود الـ Partition + قيم قريبة من المنتصف — فالأخطاء تتمركز عند الحدود.

ب) Black-Box vs White-Box Testing

النوع	المصدر	المعرفة المطلوبة
Black-Box	من المواصفة (Specification)	لا تحتاج معرفة الكود الداخلي
White-Box	من الكود نفسه	تنظر داخل الكود لإضافة Exceptions Partitions

الأفضل: الجمع بين الاثنين — Black-box أساس + White-box لتكملة النقاط الفائتة.

ج) Guideline-Based Testing للتسلسلات والمصفوفات

عند اختبار برامج تعمل على **Sequences / Arrays / Lists**:

1. اختبر تسلسل يحتوي عنصر واحد فقط — المبرمجون قد يفترضون عدة عناصر دائماً.
2. استخدم تسلسلات بأحجام مختلفة في اختبارات مختلفة.
3. اختبر العنصر الأول + الأوسط + الأخير في التسلسل.

إرشادات Whittaker العامة (2009):

- أجبر النظام على توليد كل رسائل الخطأ.
- صمّم مدخلات تُسبب تجاوز الـ Buffer.
- كرر نفس المدخل مرات عديدة.
- أجبر النظام على توليد مخرجات غير صالحة.
- أجبر نتائج الحسابات أن تكون كبيرة جداً أو صغيرة جداً.

---

3 — Component Testing

Component Testing = اختبار مجموعة وحدات مدمجة معاً تُشكّل مكوّناً — التركيز على الواجهات (**Interfaces**) بين الوحدات.

✅ يفترض أن Unit Testing على الوحدات الفردية قد اكتمل.

4 أنواع واجهات (Interfaces):

النوع	الوصف	مثال
Parameter Interfaces	تمرير بيانات أو مراجع دوال بين المكوّنات	طرق الكائن Object Methods
Shared Memory Interfaces	كتلة ذاكرة مشتركة بين المكوّنات	Embedded Systems — أجهزة الاستشعار
Procedural Interfaces	مكوّن يُغلّف مجموعة إجراءات يمكن استدعاؤها	Objects و Reusable Components
Message Passing Interfaces	طلب خدمة عبر رسالة والرد يتضمن النتيجة	Client-Server Systems

3 أنواع أخطاء الواجهات:

نوع الخطأ	الوصف	مثال
Interface Misuse	المُستدعي يُخطئ في استخدام الواجهة	نوع خاطئ للمعامل / ترتيب خاطئ / عدد خاطئ
Interface Misunderstanding	المُستدعي يُسيء فهم سلوك الواجهة	استدعاء Binary Search بمصفوفة غير مُرتَّبة
Timing Errors	في Real-Time Systems — المُنتِج والمُستهلِك بسرعات مختلفة	الـ Consumer يقرأ بيانات قديمة لأن الـ Producer لم يُحدِّثها بعد

5 إرشادات اختبار الواجهات:

1. اختبر القيم الحدية (Extreme values) لمعاملات المكوّنات الخارجية.
2. اختبر الواجهات دائماً بـ Null Pointer عند تمرير مؤشرات.
3. صمّم اختبارات تُسبّب فشل المكوّن عمداً لفحص افتراضات الفشل.
4. استخدم Stress Testing في أنظمة Message Passing لكشف مشاكل التوقيت.
5. في Shared Memory — غيّر ترتيب تفعيل المكوّنات لكشف الافتراضات الضمنية.

System Testing — 4

System Testing = دمج المكوّنات معاً واختبار النظام كوحدة واحدة — التركيز على التفاعلات بين المكوّنات.

الفرق الجوهري بين Component Testing و System Testing:

المعيار	Component Testing	System Testing
المدى	مكوّنات فردية مدمجة	النظام كاملاً
المكوّنات	فقط ما طوّره الفريق	COTS + Reusable Components +
الطابع	فردي	جماعي (Collective)
الفريق	المطوّر نفسه	قد يكون فريق اختبار مستقل

Emergent Behavior — السلوك الظاهر

النظام يُظهر سلوكاً لا يظهر إلا عند دمج المكوّنات:

- **Planned Emergent Behavior** = سلوك متوقع يجب اختباره (مثل: دمج Authentication مع Database Update).
- **Unplanned Emergent Behavior** = سلوك غير مرغوب يجب اكتشافه وإزالته.

Use Case-Based Testing — الأسلوب المثالي لـ System Testing

كل Use Case يُجبر عدة مكوّنات على التفاعل معاً. إذا لديك **Sequence Diagram** ← استخدمه لتحديد العمليات وتصميم حالات الاختبار.

متى نتوقف عن System Testing؟

Exhaustive Testing = مستحيل — الاختبار يجب أن يعتمد على سياسات مثل:

1. كل دوال القوائم (Menus) تُختبر.
2. تركيبات الدوال في نفس القائمة تُختبر معاً.
3. كل دالة تُختبر بمدخلات صحيحة وخاطئة.

⚠ **الأخطاء الشائعة في System Testing:** الدوال منفردة تعمل جيداً لكن تركيبات الدوال النادرة لم تُختبر معاً — مثال Whittaker: الـ Footnotes مع Multicolumn Layout في برامج النصوص!

نقاط MCQ احفظها جيداً

- ◈ **Development Testing** = يُجريه فريق التطوير — هدفه Defect Testing.
- ◈ **3 مستويات:** Unit ← Component ← System.
- ◈ **Unit Testing** = اختبار الوحدات الفردية (دوال + كلاسات).
- ◈ **3 أشياء تُغطيها في Unit Test:** جميع العمليات + الخصائص + الحالات.
- ◈ **الوراثة تُعقّد Unit Testing** ← اختبر العملية في كل كلاس فرعي يرثها.
- ◈ **3 أجزاء الاختبار التلقائي:** Setup + Call + Assertion.
- ◈ **Mock Objects** = بنفس Interface الكائن الحقيقي — تُحاكي السلوك بدون overheads.
- ◈ **JUnit** = أداة الاختبار التلقائي لـ Java.
- ◈ **2 استراتيجيتان لاختيار Test Cases:** Partition Testing + Guideline-Based Testing.
- ◈ **Equivalence Partitioning** = مجموعات بنفس السلوك ← اختبر من كل مجموعة.
- ◈ **Boundary Values** = الأهم في الاختبار — الأخطاء تتمركز عندها.
- ◈ **Black-Box** = من المواصفة | **White-Box** = من الكود.
- ◈ **4 أنواع Interfaces:** Parameter + Shared Memory + Procedural + Message Passing.
- ◈ **3 أنواع Interface Errors:** Misuse + Misunderstanding + Timing Errors.
- ◈ **Timing Errors** = تحدث في Real-Time Systems مع Shared Memory أو Message Passing.
- ◈ **System Testing** = جماعي (Collective) ≠ Component Testing = فردي.
- ◈ **Emergent Behavior** = سلوك يظهر فقط عند دمج المكوّنات.
- ◈ **Use Case-Based Testing** = أفضل أسلوب لـ System Testing.
- ◈ **Exhaustive Testing** = مستحيل تطبيقياً.

Test-Driven Development (TDD)

تعريف TDD

Test-Driven Development (TDD) = نهج في تطوير البرامج تكتب فيه **الاختبارات أولاً قبل كتابة الكود** — يتشابك فيه التطوير والاختبار معاً بشكل تدريجي (Incremental).

نشأته: قُدّم أصلاً كجزء من **XP (Extreme Programming)** — أحد أساليب Agile — ثم أصبح منهجاً رئيسياً يُستخدم في كل من Agile و Plan-Based processes.

خطوات عملية 5 — TDD خطوات دائرية

1. **حدّد الوظيفة الصغيرة المطلوبة**
   ↓
2. **اكتب اختباراً تلقائياً لها**
   ↓
3. **شغّل الاختبار ← سيفشل** ✗ (لأن الكود لم يُكتب بعد)
   ↓
4. **نفّذ الكود + أعد هيكلته (Refactor)**
   ↓
5. **شغّل الاختبار مجدداً ← يجب أن ينجح** ✓
   ↓

انتقل للوظيفة التالية وكرّر الدورة

⚠ **لماذا الفشل المتعمد في الخطوة 3 مهم؟** — لإثبات أن الاختبار الجديد يُضيف شيئاً فعلياً للمجموعة، ولم يكن موجوداً من قبل

الأداة المطلوبة لـ TDD

JUnit = إطار العمل الأساسي لتشغيل الاختبارات التلقائية في Java — يسمح بتشغيل **مئات الاختبارات في ثوانٍ** مع كل تغيير صغير في الكود.

كيف يساعد TDD على فهم المشكلة؟

لكتابة اختبار، يجب أن تفهم **بدقة** ما يجب على الكود فعله — هذا الفهم يجعل كتابة الكود نفسه أسهل.

مثال من الكتاب: إذا كان الكود يحتوي عملية قسمة ← يجب كتابة اختبار يتحقق من **القسمة على صفر**. إذا نسيت هذا الاختبار، لن يُكتب الكود الوقائي أبداً.

✨ فوائد TDD الأربع (الأهم للامتحان)

الفائدة	الشرح التفصيلي
Code Coverage (تغطية الكود)	كل جزء من الكود له اختبار ← ثقة بأن كل الكود تم تنفيذه فعلاً، والأخطاء تُكتشف مبكراً
Regression Testing (الاختبار الانحداري)	مجموعة اختبارات تنمو مع النظام ← يمكن إعادة تشغيلها في أي وقت للتحقق من عدم ظهور أخطاء جديدة بعد التعديلات
Simplified Debugging (تبسيط تصحيح الأخطاء)	عند فشل اختبار، الخطأ واضح المكان — الكود الجديد فقط هو المشكوك فيه — **نادراً ما تحتاج Debugger**
System Documentation (توثيق النظام)	الاختبارات نفسها تعمل كـ **وثائق حية** تشرح ما يجب على الكود فعله

🔁 Regression Testing — لماذا هو الفائدة الأهم؟

Regression Testing = إعادة تشغيل الاختبارات بعد كل تعديل للتأكد من أن التغييرات لم تُدخل أخطاء جديدة.

المعيار	اختبار يدوي	TDD (تلقائي)
التكلفة	عالية جداً — وقت وجهد	منخفضة جداً
الاختيار	يجب اختيار الأهم — قد يفوتك شيء	كل الاختبارات تُشغَّل دائماً
قاعدة TDD	----------	قبل إضافة أي وظيفة جديدة ← يجب أن **تنجح كل الاختبارات السابقة**

⛔ قيود TDD ومتى لا يعمل

يُقدّم الكتاب **4 حالات** يكون فيها TDD غير فعّال أو غير عملي:

الحالة	السبب
Legacy Systems (الأنظمة القديمة)	لا يمكن تفكيكها لعناصر صغيرة قابلة للاختبار
مكوّنات كبيرة معاد استخدامها	يجب اختبارها ككتلة واحدة كاملة
Multithreaded Systems	الخيوط قد تتداخل بترتيبات مختلفة في كل تشغيل ← نتائج متناقضة
نقص المعرفة	إذا لم تفهم المطلوب كفايةً لكتابة اختبار ← لن تكتب الكود الصحيح أصلاً

⚠ TDD لا يُغني عن System Testing

حتى مع استخدام TDD، **لا تزال تحتاج** إلى عملية System Testing لأن:

- تحتاج **Validation** = التحقق من أن النظام يلبي متطلبات **جميع** أصحاب المصلحة
- TDD لا يختبر **الأداء (Performance)** و **الموثوقية (Reliability)**
- TDD لا يتحقق من أن النظام **لا يفعل أشياء غير مطلوبة** (Unwanted outputs)

TDD في سياق Agile

الجانب	التفصيل
الأصل	XP (Extreme Programming)
الانتشار	أصبح سائداً في كل من Agile و Plan-Based
في Agile (XP)	لا يوجد Acceptance Testing منفصل — المستخدم المدمج في الفريق يكتب الاختبارات كـ User Stories
ادّعاء مهم	TDD يُشجّع على **هيكلة أفضل للكود** و **جودة أعلى** — لكن التجارب لم تُثبت هذا الادعاء قطعياً

نقاط MCQ احفظها جيداً

- ◈ **TDD** = اكتب الاختبار **أولاً** ثم اكتب الكود
- ◈ **نشأ** مع XP (Extreme Programming) ← نوع من Agile
- ◈ **الاختبار يفشل أولاً** بشكل متعمد ← هذا صحيح ومقصود ☑
- ◈ **5 خطوات:** حدّد الوظيفة ← اكتب اختباراً ← شغّله (فشل) ← نفّذ الكود ← شغّله (نجاح)
- ◈ **الأداة:** JUnit لـ Java
- ◈ **4 فوائد:** Code Coverage + Regression Testing + Simplified Debugging + Documentation
- ◈ **الفائدة الأهم:** تقليل تكلفة **Regression Testing**
- ◈ **TDD لا يصلح مع:** Legacy Systems + Multithreaded Systems + Reused Large Components
- ◈ **TDD لا يُغني عن System Testing** ← لا يختبر Performance ولا Reliability
- ◈ في **XP** ← المستخدم المدمج يكتب اختبارات تعادل **Acceptance Tests**
- ◈ **Regression Testing** = إعادة تشغيل الاختبارات بعد كل تعديل للتحقق من عدم ظهور أخطاء جديدة

Release Testing (اختبار ما قبل الإصدار)

تعريف Release Testing

Release Testing = عملية اختبار **إصدار معين** من النظام **مُعدّ للاستخدام خارج فريق التطوير** — سواء كان للعملاء، أو لفرق أخرى داخل المشروع، أو لإدارة المنتج.

الفرق الجوهري بين Release Testing و System Testing

المعيار	System Testing	Release Testing
من يُجريه؟	فريق التطوير نفسه	فريق **مستقل** — لا يشمل المطورين
الهدف	اكتشاف الأخطاء (Defect Testing)	التحقق من الجودة للعميل (Validation)
المنهج	يعرف الكود الداخلي	**Black-Box** فقط — يعتمد المواصفة
اسم آخر له	—	Functional Testing

الهدف الأساسي لـ Release Testing: إقناع **مورّد النظام** بأنه جيد كفايةً للاستخدام وإطلاقه كمنتج أو تسليمه للعميل.

هيكل القسم — 3 أساليب

- **8.3.1 Requirements-Based Testing** ← اختبار بناءً على المتطلبات
- **8.3.2 Scenario Testing** ← اختبار السيناريوهات الواقعية
- **8.3.3 Performance Testing** ← اختبار الأداء والإجهاد

---

1 — (8.3.1) Requirements-Based Testing

المبدأ: كل متطلب يجب أن يكون **قابلاً للاختبار** ← صمّم مجموعة اختبارات لكل متطلب بعينه.

نوعه: Validation Testing — أنت تُثبت أن النظام نفّذ المتطلبات — لا تبحث عن أخطاء.

مثال تطبيقي من نظام Mentcare — اختبار متطلب التحذير من الحساسية:

المتطلب: إذا كان المريض يعاني من حساسية لدواء معين، فإن وصف ذلك الدواء يجب أن يُصدر رسالة تحذير للمستخدم.

الاختبارات المطلوبة (5 اختبارات لمتطلب واحد!):

الاختبار	ما يتحقق منه
مريض بلا حساسية ← وصف دواء	لا يظهر تحذير ✅
مريض لديه حساسية واحدة ← وصف نفس الدواء	يظهر تحذير ✅
مريض لديه حساسيتان ← وصف كل دواء بشكل منفصل	ظهور التحذير الصحيح لكل دواء ✅
وصف دواءين معًا والمريض حساس لهما	ظهور تحذيرين ✅
وصف دواء + تجاهل التحذير	يطلب النظام سبب التجاهل ✅

💡 **درس مهم:** متطلب **واحد** يحتاج إلى **عدة اختبارات** — ليس اختباراً واحداً فقط.

🔔 يجب الاحتفاظ بـ **Traceability Records** تربط كل اختبار بالمتطلب الذي يختبره.

---

2 — Scenario Testing

Scenario Testing = أسلوب يعتمد على **قصص استخدام واقعية** لاستخلاص حالات الاختبار — يحاكي طريقة استخدام النظام الحقيقية.

خصائص السيناريو الجيد (حسب Kaner 2003):

- **Credible** — قصة موثوقة وواقعية.
- **Complex** — معقد بما يكفي لاختبار تفاعلات متعددة.
- **Motivating** — يجعل أصحاب المصلحة يهتمون به ويؤمنون بأهميته.
- **Easy to evaluate** — سهل تقييم نجاحه أو فشله.

مثال تطبيقي — سيناريو الزيارة المنزلية في نظام Mentcare:

جورج ممرض متخصص في الصحة النفسية ← يزور المرضى في منازلهم ← يسجّل ملاحظاته عبر اللابتوب ← يُحدّث السجلات عند العودة للعيادة.

هذا السيناريو الواحد يختبر 7 ميزات في آن واحد:

1. المصادقة وتسجيل الدخول (Authentication)
2. تحميل ورفع سجلات المرضى للابتوب
3. جدولة الزيارات المنزلية
4. تشفير وفك تشفير السجلات على الأجهزة المتنقلة
5. استرجاع وتعديل السجلات
6. الربط بقاعدة بيانات الأدوية والآثار الجانبية
7. نظام تذكير المكالمات

☆ **الميزة الكبرى لـ Scenario Testing:** في اختبار واحد تختبر **تركيبات متعددة من المتطلبات** معاً — وليس كل متطلب بشكل مستقل.

💡 **تلميح عملي:** المختبِر يلعب دور الشخصية في السيناريو ويُدخل أخطاءً **متعمدة** (مثل كلمة سر خاطئة) لاختبار استجابة النظام.

(8.3.3) Performance Testing — 3

Performance Testing = اختبار يُجرى **بعد اكتمال دمج النظام** للتحقق من الخصائص الظاهرة (Emergent Properties) كالأداء والموثوقية.

أ) Operational Profile

Operational Profile = مجموعة اختبارات تعكس **مزيج العمل الحقيقي** للنظام.

- **مثال:** إذا كان 90% من المعاملات من النوع A، و5% من النوع B، والباقي C وD وE ← يجب أن تكون **الغالبية العظمى من الاختبارات من النوع A** للحصول على قياس دقيق للأداء التشغيلي.

ب) Stress Testing — الأهم للامتحان

Stress Testing = اختبار النظام بأحمال **تتجاوز حدود التصميم** لاكتشاف الأخطاء ومراقبة سلوك الفشل.

هدفا Stress Testing:

الهدف	التفصيل
اختبار سلوك الفشل	لا يُفسد البيانات ولا يقطع الخدمات فجأة — (Fail-Soft) النظام يجب أن يفشل بهدوء
كشف أخطاء تظهر فقط تحت الحمل الكامل	يكشفها **Stress Testing** بعض الأخطاء لا تظهر في الاستخدام العادي، لكن

لماذا Stress Testing مهم خاصة للأنظمة الموزعة؟

الأنظمة الموزعة على شبكة معالجات تُعاني من **تدهور حاد** عند الحمل الزائد:

1. الشبكة تمتلئ ببيانات التنسيق بين العمليات.
2. العمليات تتباطأ في انتظار البيانات من العمليات الأخرى.

Stress Testing يُحدد **نقطة بداية التدهور** ليتم إضافة فحوصات ترفض المعاملات بعدها.

مقارنة الأساليب الثلاثة

الأسلوب	هدفهم	مصدر الاختبارات	يختبر
Requirements-Based	Validation	وثيقة المتطلبات	وظائف محددة
Scenario Testing	Validation + تركيبات	قصص استخدام واقعية	تفاعلات متعددة معاً
Performance Testing	Validation + Defect	حدود التصميم + Operational Profile	الأداء والموثوقية

نقاط MCQ احفظها جيداً

- ◈ **Release Testing** = يُجريه فريق **مستقل** — لا فريق التطوير.
- ◈ **هدفه** = Validation — إثبات أن النظام جيد للاستخدام.
- ◈ **منهجه** = **Black-Box** دائماً — يُسمى أيضاً **Functional Testing**.
- ◈ **متطلب واحد** يحتاج إلى **عدة اختبارات** — ليس اختباراً واحداً.
- ◈ **Traceability Records** = سجلات تربط الاختبارات بمتطلباتها.
- ◈ **السيناريو الجيد** = واقعي + معقد + محفّز + سهل التقييم (Kaner 2003).
- ◈ **Scenario Testing** يختبر **تركيبات متعددة** من المتطلبات في اختبار واحد.
- ◈ **Performance Testing** يُجرى بعد **اكتمال دمج النظام** كاملاً.
- ◈ **Operational Profile** = اختبارات تعكس مزيج العمل **الحقيقي** للنظام.
- ◈ **Stress Testing** = تجاوز حدود التصميم لكشف سلوك الفشل.
- ◈ **Fail-Soft** = النظام يفشل بهدوء بدون تلف البيانات أو انقطاع الخدمات.
- ◈ **Stress Testing مهم** خصوصاً في **Distributed Systems** (الأنظمة الموزعة).
- ◈ السيناريوهات المُستخدمة في **Requirements Engineering** يمكن **إعادة استخدامها** في Scenario.

User Testing (اختبار المستخدم)

تعريف User Testing

User Testing = مرحلة في عملية الاختبار يُقدّم فيها **المستخدمون أو العملاء** المدخلات والمشورة حول النظام — سواء كان ذلك اختباراً رسمياً لنظام مُشترى أو تجربة غير رسمية للتحقق من الجودة.

لماذا لا يمكن الاستغناء عنه؟ لأن المطوّر **لا يستطيع** أبداً محاكاة بيئة العمل الحقيقية — المستشفى مثلاً يضم حالات طوارئ ومحادثات تؤثر على الاستخدام.

هيكل القسم — 3 أنواع للأختبارات

- **Alpha Testing** ← مستخدمون مختارون + فريق التطوير
- **Beta Testing** ← جمهور أوسع + إصدار مبكر
- **Acceptance Testing** ← العميل الرسمي ← القرار النهائي

---

1 — Alpha Testing

Alpha Testing = مجموعة **مختارة** من المستخدمين تعمل **جنباً إلى جنب مع فريق التطوير** لاختبار إصدارات مبكرة من البرنامج.

خصائصه:

- يُستخدم كثيراً في تطوير **Software Products** و**Apps**.
- المستخدمون الخبراء يشاركون لأن ذلك يمنحهم **معلومات مبكرة** عن المزايا الجديدة.
- يُقلّل من مخاطر التغييرات المفاجئة التي تُربك أعمالهم.
- في **Agile**: المستخدم المدمج في الفريق يلعب دور **Alpha Tester** ويُصمّم الاختبارات.

---

2 — Beta Testing

Beta Testing = إتاحة **إصدار مبكر** (أحياناً غير مكتمل) أمام **قاعدة أوسع** من المستخدمين للتقييم والاكتشاف.

خصائصه:

الجانب	التفصيل
الجمهور	مستخدمون أوائل مختارون (Early Adopters) — أو الجمهور العام بالكامل
الاستخدام الرئيسي	منتجات تعمل في بيئات متنوعة لا يمكن للمطوّر محاكاتها
ما يكتشفه	تعارضات بين البرنامج وخصائص بيئات تشغيل مختلفة
ميزة إضافية	يعمل كأداة تسويقية؛ حيث يتعرّف العملاء على النظام وقدراته

🔔 **الفرق الجوهري بين Alpha وBeta:**

المعيار	Alpha	Beta
الحجم	مجموعة صغيرة مختارة	جمهور أوسع / عام
العلاقة بالفريق	تعاون مباشر مع فريق التطوير	مستقل — يُبلّغ عن المشاكل
مرحلة النظام	إصدارات مبكرة أثناء التطوير	إصدار مبكر شبه مكتمل
الهدف	تحسين التصميم والاختبارات	اكتشاف مشاكل البيئات المتنوعة

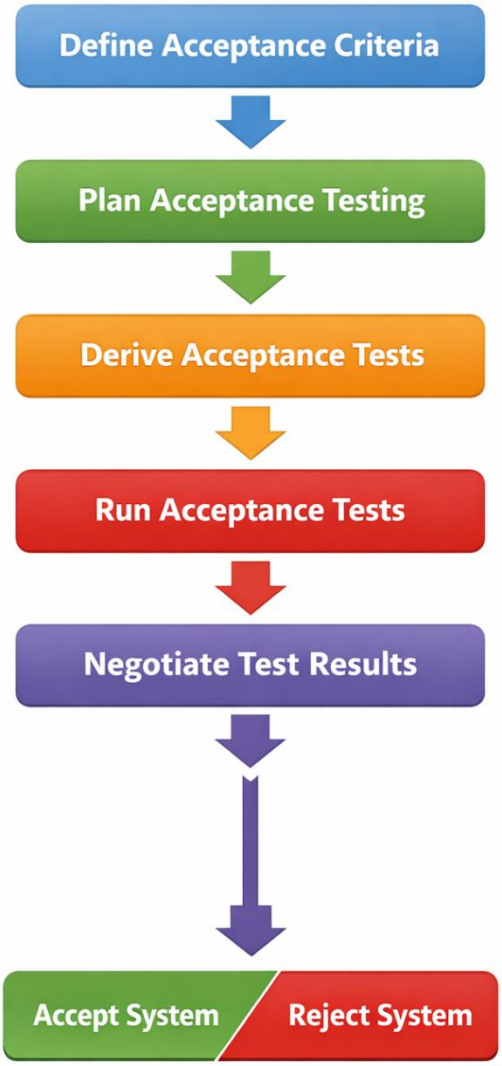
---

3 — Acceptance Testing (الأهم للامتحان!)

Acceptance Testing = اختبار **رسمي** يُجريه العميل **ببياناته الخاصة** ليقرر **قبول أو رفض** تسليم النظام من المطوّر.

🔔 **القبول = الموافقة على الدفع النهائي** للمطوّر.

الـ 6 مراحل الرسمية لـ Acceptance Testing (Figure 8.11):



تفصيل الـ 6 مراحل:

#	المرحلة	التفصيل
1	**Define Acceptance Criteria**	يجب تحديدها **مبكراً قبل التوقيع على العقد** — تُصبح جزءاً من العقد ومعتمدة من العميل والمطوّر — لكن تحديدها مبكراً صعب لأن المتطلبات قد تتغير
2	**Plan Acceptance Testing**	تحديد الموارد + الوقت + الميزانية + جدول الاختبارات + تغطية المتطلبات + ترتيب الاختبار + تحديد المخاطر (انهيار النظام، ضعف الأداء)
3	**Derive Acceptance Tests**	تصميم اختبارات تُغطي **الوظائف والخصائص غير الوظيفية** (Performance) — يجب أن تُغطي جميع المتطلبات تقريباً — صعب وضع معايير موضوعية تماماً
4	**Run Acceptance Tests**	تنفيذ الاختبارات المتفق عليها — الأفضل في **البيئة الحقيقية** لكن قد يكون مُربِكاً — قد تُطلب بيئة اختبار خاصة — قد يحتاج المستخدمون تدريباً
5	**Negotiate Test Results**	نادراً ما تنجح جميع الاختبارات — العميل والمطوّر يتفاوضان على: هل النظام جيد بما يكفي؟ وكيف تُحَل المشاكل المكتشفة؟
6	**Accept or Reject System**	اجتماع رسمي للقرار النهائي — إذا رُفض: يعود للتطوير ثم يُعاد الاختبار — إذا قُبِل: يُنشر النظام

واقعية Acceptance Testing — ليس أبيض أو أسود!

قد يقبل العميل النظام رغم وجود مشاكل لأن **تكلفة عدم الاستخدام أكبر من تكلفة المشاكل** — فالعميل اشترى أجهزة جديدة وأبّل موظفيه وغيّر عملياته!

النتيجة الشائعة: Conditional Acceptance = قبول مشروط — يُنشر النظام مع التزام المطوّر بإصلاح المشاكل العاجلة وتسليم إصدار جديد بسرعة.

---

4 — Acceptance Testing في Agile / XP

في **Agile (XP)** لا يوجد Acceptance Testing منفصل — بدلاً من ذلك:

الجانب	ما يحدث في XP
المستخدم	مدمج في فريق التطوير كـ Alpha Tester
المتطلبات	تُكتب كـ User Stories
الاختبارات	المستخدم نفسه يُصمّمها لتحديد قبول/رفض الـ User Story
طبيعتها	تلقائية (Automated) — لا يُكمَل التطوير حتى تنجح
مكافئها	تعادل **Acceptance Tests** في النماذج التقليدية

⚠ **مشكلة المستخدم المدمج في Agile:**

- يجب أن يكون **مستخدماً نموذجياً** — صعب إيجاده.
- الاختبارات التلقائية **تُقيّد** اختبار الأنظمة التفاعلية.
- قد لا تعكس الاختبارات واقع الاستخدام الحقيقي.

- **الحل الشائع:** مزج Agile مع Acceptance Testing منفصل بعد الإصدار الكبير.

مقارنة الأنواع الثلاثة — جدول شامل

المعيار	Alpha	Beta	Acceptance
من يختبر؟	مستخدمون مختارون	جمهور أوسع / عام	العميل الرسمي
متى؟	أثناء التطوير	إصدار مبكر	بعد Release Testing
البيئة	مع فريق التطوير	بيئة المستخدم الفعلية	بيئة العميل الفعلية
الهدف	تحسين التصميم والاختبارات	اكتشاف مشاكل البيئات	قرار القبول/الرفض
رسمي؟	لا	لا	نعم — عقدي
نتيجته	ملاحظات + تحسينات	تقارير أخطاء	قبول / رفض / قبول مشروط

نقاط MCQ احفظها جيداً

- ◈ **User Testing** ضروري حتى بعد Release Testing — البيئة الحقيقية لا يمكن محاكاتها.
- ◈ **3 أنواع:** Alpha + Beta + Acceptance.
- ◈ **Alpha** = مجموعة صغيرة + مع فريق التطوير + إصدارات مبكرة.
- ◈ **Beta** = جمهور أوسع + مستقل + لاكتشاف مشاكل البيئات المتنوعة.
- ◈ **Beta** = أداة تسويق أيضاً ← العملاء يتعرفون على النظام.
- ◈ **Acceptance Testing** = رسمي + عقدي + **العميل بياناته الخاصة**.
- ◈ **القبول = الموافقة على الدفع النهائي**.
- ◈ **6 مراحل Acceptance:** Define Criteria → Plan → Derive Tests → Run → Negotiate → Accept/Reject.
- ◈ **Conditional Acceptance** = قبول مشروط مع التزام بإصلاح المشاكل العاجلة.
- ◈ في **Agile/XP** = لا Acceptance Testing منفصل — المستخدم المدمج يكتب اختبارات User Stories.
- ◈ اختبارات **XP** هي مكافئ **Acceptance Tests** التقليدية.
- ◈ **مشكلة Agile:** صعوبة إيجاد مستخدم نموذجي + قيود الأتمتة على الأنظمة التفاعلية.
- ◈ **الحل الشائع:** مزج Agile + Acceptance Testing منفصل بعد كل إصدار كبير.